Gauge reader v1 service report: what we built and how it differs from v0

CarbonMint water-gauge reader · Oct 12, 2026

v1 reads the water level of a paddy-field gauge from a drone photo. It is packaged as a Docker container: the client drops in a zip of photos and gets a JSON report back. On the 127 pilot photos, v1 gives an accepted, exactly correct stripe count on **97 photos (76.4%)**. The existing reader, v0, manages **19 (15.0%)**. The target was 70 to 80%.

15.0%	76.4%	28 → 16
v0: accepted and exact (19 of 127)	v1: accepted and exact (97 of 127)	accepted but wrong, v0 → v1

What we did

The client flies a drone over each paddy field every day and photographs a small float gauge standing in the water. As the water rises, red stripes come up out of a black sleeve, about one stripe per centimetre. No stripes showing means the field is dry. The crop often grows over the gauge, and the client's existing reader (v0) gets only about half of the photos right. We were asked to reach 70 to 80% without changing the gauge itself.

1. **We made an answer key.** For every pilot photo we marked the first and last stripe and counted the stripes by hand, as whole stripes. A person then checked two random samples of 15 against the photos: the first round matched 10 of 15, so the key was corrected; the second round matched 15 of 15. Every number in this report is checked against that key.
2. **We rebuilt v0 and measured it.** We did not have v0's code, only a written description of how it works, so we rebuilt it rule by rule from that description and ran it on the same photos.
3. **We designed v1.** Instead of counting stripes one by one, v1 measures the rhythm of the stripes along the gauge, so stripes hidden by a leaf still count. A small trained model then judges how likely the count is to be exactly right, and the client's own acceptance rule decides. We tried each idea separately and kept only what helped.
4. **We checked the weak spots.** We tested enlarging far-away gauges, counting an extra stripe at the sleeve, and several other ideas. Those that did not improve the result on fields the model had not seen were dropped.
5. **We packaged it as a service.** A Docker container that watches a folder: a zip of photos goes in, a JSON report comes out.
6. **We tested the container with the real zip** (127 photos, 713 MB) and confirmed it gives the same answers as our research code on every photo.

How the service works

The container runs all the time. The client copies a zip into one folder and collects the report from another.

1. **A zip goes into the `in` folder.** It should look like the first zip the client sent: one folder per visit, named `<checkpoint> - <ddmmyyHHMMSS>`, with that visit's photos inside. The folder name tells us the checkpoint and the time, and keeps the two shots of one visit together.
2. **The container notices it.** It looks at the folder every 10 seconds and waits until the file size stops changing, so it never reads a zip that is still being copied.
3. **It reads the photos directly from the zip.** Nothing is unpacked onto the disk. Photos are processed one visit at a time, so the two shots of a visit can check each other.
4. **Each photo goes through v1** (next sections).
5. **A report is written** to the `out` folder as `<zipname>.report.json`, and the zip is moved to `in/done`. A zip that cannot be read goes to `in/failed`, with an error file saying why. One damaged photo never stops the batch; it is marked as an error and the rest carry on.
6. **The same zip dropped twice is only processed once.** The container recognises it by its fingerprint (SHA-256), and remembers it across restarts.

For safety, file names that try to climb out of the folder are ignored, only jpg and png files are read, very large single files are skipped, and there is a limit on how many photos one zip may hold.

What the report contains

The report is a JSON file, plain text that any system can read. It has a summary, then one entry per visit, then one entry per photo. A schema file describes exactly what a valid report looks like.

Field	Meaning
`status`	`accepted`, `review` (not sure enough, so no number is given), `no_gauge` or `error`
`stripes`	The stripe count, filled in only for accepted photos. `0` with `dry: true` means the gauge is down and the field is dry
`candidate_stripes`	For a photo in review: the count a person should check
`confidence`	The chance that the count is exactly right; a photo is accepted at 0.7 or more
`level_cm`, `soil_state`	The water level in cm and a label: Flooded, Saturated, Drying or Irrigate now
`gauge_box`	Where the gauge was found in the photo
`flags`	Anything unusual: no second shot of the visit, the second shot unreadable, the colour locator used, a bad gauge
visit `reading`	The mean of the visit's accepted photos (the client's folder rule), and whether they agree

The soil state follows v0's own rule: the guide zero sits 10 cm up the gauge. Above +2 cm is Flooded, −2 to +2 is Saturated, down to −12 is Drying, and below that is Irrigate now. A real entry from the pilot zip, trimmed:


```
{
  "file": ".../AIT-CP-5286-2931-CKP K26 NG - 011026101600/dji_fly_20261001_101600_0358_..._photo.jpg",
  "status": "accepted",
  "stripes": 4,
  "dry": false,
  "level_cm": 4.0,
  "level_vs_guide_cm": -6.0,
  "soil_state": "Drying",
  "confidence": 0.934,
  "gauge_box": [2095, 969, 2236, 1071],
  "flags": []
}
```

How one photo is read

v0 looks for the pinkest column and counts the short pink bars in it. v1 measures instead of counting, then asks how sure it is.

Step	What it does	Why it is there
1. Find the gauge	Looks for a stack of separate red stripes on one near-upright line, judging red against green and blue rather than by fixed brightness. A gauge split by a leaf is joined back up	Shaded stripes are dark; a fixed brightness bar misses them
2. Straighten it	Follows the gauge's lean and bend through the stripe centres	Drone shots are rarely square-on
3. Read the stripe rhythm	Measures the spacing of the stripes and counts gap by gap from the first stripe to the last	Stripes hidden by a leaf still count; perspective changes the spacing along the gauge
4. The colour locator	If step 1 finds nothing, our colour method takes over: it evens out brightness, switches off the green crop, and finds the gauge by its stripes, blue-grey pipe and black sleeve	A dry gauge has no stripes for step 1 to find
5. How sure it is	A small trained model gives the chance the count is exactly right, from the strength of the rhythm, how many stripes were actually seen, sharpness, gauge size and whether the visit's other shot agrees	The 0.7 bar then means what it says: probably exact
6. Client rules	Accept at 0.7; a visit takes the mean of its accepted photos; a reading that breaks the gauge's physics is flagged as a bad gauge and never accepted	The same rules v0 follows

A dry gauge is a reading. When the gauge is found but no stripes are showing, v1 reports 0 stripes and `dry: true`. This is the reading AWD irrigation needs most, and v0 cannot give it. Of the 29 dry photos, v1 reads 28 correctly.

When it is unsure, the photo is marked `review`. We give no number rather than guess, so a person can check it.

What the service is built from

Everything is free, open-source software, and nothing needs a paid service or an internet connection once it is installed. The container runs as a normal user rather than an administrator, restarts itself if it stops, and is limited to 2 GB of memory.

Part	What it is	What we use it for
Docker	A way of packaging a program with everything it needs into one sealed box	The client runs the same box on any machine, with nothing else to install
Python 3.11	The programming language	All the logic: reading the zip, reading the gauges, writing the report
NumPy and SciPy	Libraries for fast calculations on large grids of numbers	A photo is a grid of numbers: finding the stripes, straightening, measuring the rhythm, and the confidence model
Pillow	A standard library for opening images	Reading the photos straight from the zip
OpenCV	A standard image-processing library	The colour locator for gauges the main finder cannot see
Confidence model	A small trained model (one file, under 1 KB)	The chance that a count is exactly right
JSON and JSON Schema	The report format, and a file describing a valid report	Any system can read the report, and reports are checked against the schema in our tests
Folder watcher	A short loop of our own code	Checks the input folder every 10 seconds and waits for copies to finish
Health check	A status signal the container gives Docker	Docker can see whether the service is alive
Docker Compose	A small settings file	Sets the folders, the memory limit, and options such as the poll interval and centimetres per stripe

How v1 differs from v0

	v0	v1
Approach	Finds the pinkest column, counts the bars in it	Measures the stripe rhythm from the first stripe to the last
How sure it is	Judged by how regular the stripes look	A trained estimate of the chance the count is exactly right
Shade	Needs bright pink stripes, so shaded ones are missed	Judges red against green and blue, so dark stripes still count
A leaf across the gauge	The count breaks where the leaf cuts the stripes	Hidden stripes are counted from the rhythm
A dry gauge	No stripes means no reading at all	Dry (0 stripes) is a proper reading
The second shot of a visit	Not used	Agreement with it raises confidence
A broken gauge	Not checked	Too many stripes, or a spacing that doesn't fit the pipe, is flagged
An unsure photo	Left out	Sent to a person, with the count to check

Results, scored the same strict way for both. A photo counts only when it is accepted and its stripe count matches the hand count **exactly**. All 127 photos are scored; none are left out. The visibility sets were sorted by eye.

Photos	v0	v1
Clear (59)	16 = 27.1%	45 = 76.3%
Partly hidden (43)	3 = 7.0%	32 = 74.4%
Hard (25)	0 = 0%	20 = 80.0%
All (127)	**19 = 15.0%**	**97 = 76.4%**
Accepted	47	113
Accepted but wrong	28	16
Visits with an accepted, exact reading (66)	9	50

v0 does accept about 60% of the clear photos (36 of 59), which matches the client's “about half”. The trouble is that more than half of what it accepts is wrong (28 of 47): its confidence measures how tidy the stripes look, not whether the count is correct.

Why v0 misses the rest (108 photos). It cannot find the gauge or a stripe stack, mostly because shaded stripes are too dark for its colour test: 37 photos. A dry gauge has no stripes, so it gives no reading: 29. It accepts a wrong count, mostly where leaves break its run of stripes: 28. A wrong count it does not accept: 13. The right count just under its confidence bar: 1.

> Scored more loosely, the way some pilot results are reported (within 1 stripe, or within 3 on tall stems counted by estimate, with photos that can't be counted by eye left out), v1 scores 94 of 118 (79.7%). This report uses the strict, exact rule throughout.

v0 here is our rebuild from its written description, because the original code was not available to us. v1's confidence model is trained on six fields and scored on the seventh, for each field in turn, so no field is scored by a model that saw it.

How we tested it

- **Same answers as the research code.** Run over the 127 photos, the container gives the same stripe count and the same accept decision as our research code on **all 127**.
- **Automated tests.** 39 checks in total. The service tests cover reading zips (visit folders, damaged photos, hostile file names, empty and broken zips, loose photos with no visit folder, too many photos), the soil-state rules, the visit rule, the report format against its schema, and the folder watcher (partly copied zips, duplicates, broken zips, restarts). The reader tests cover synthetic gauges with known answers, hidden stripes, dry gauges, the confidence model and the bad-gauge check.
- **The real zip, end to end.** We started the container, copied in the real 127-photo zip (713 MB) and waited. The report appeared about 4.5 minutes later (267 seconds of reading), the zip moved to `done`, and the container reported healthy: 111 photos accepted, 16 sent to review, none with no gauge found, none in error. Memory peaked at 518 MB, well inside the 2 GB limit. Dropping the same zip again was recognised and skipped.
- **A problem we found and fixed.** One dry gauge was found by our research code but not by the first build of the container: the container's colour locator skipped the brightness-evening step. With that step added, the container and the research code agree on every photo.
- **The answer key itself was checked by a person** (15 of 15 in the second round), so the scores are not measured against unchecked numbers.

Limits and next steps

Limits we want to be upfront about

- The 76.4% was measured on the photos v1 was built on. The confidence model was always scored on a field it had not seen, but the reading rules were designed looking at these photos. The client's next upload will be the real test.
- One stripe is one centimetre is an assumption. It is not confirmed with the client, and `level_cm` and `soil_state` both depend on it. It is a single setting to change once someone measures a gauge.

- A dry gauge is reported as Drying, never Irrigate now. v0's rule puts the guide zero 10 cm up the gauge, so zero stripes works out to −10 cm, which falls in the Drying band. We applied the rule as written, but it may need a second look.
- A photo with no second shot is weaker, because the visit check is missing. It is flagged in the report.
- No photo in our batch shows a faulty gauge, so the bad-gauge check has never seen a real one.
- 16 photos were accepted with a wrong count, mostly one stripe off at the top or bottom of the gauge, where a stripe is half hidden by the cap or the sleeve.
- The container's own accept decisions use the model trained on all seven fields, so on these same photos it looks slightly better (111 accepted, 98 exact) than the fair score above.

Next steps

- Run v1 on a fresh batch of client photos, including shots from farther away, and confirm the 76%.
- Ask the client for the real centimetres per stripe, or measure one gauge in the field.
- Agree with the client what the soil state should say for a dry gauge.
- Ask pilots to keep the gauge large in the frame and take two steady shots per visit; blurred and far shots are where v1 most often asks for review.
- Hand the repository and container to the client, with the service README as the starting point.